YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – Low Level Design

On

VISUAL THREAT HUNTING FOR E-COMMERCE NETWORKS

SUBMITTED BY

Muhammed Dhanish - 23BCCDD035

BCA(Cybersecurity, Digital Forensic and Data Science)

UNDER THE GUIDANCE OF

Industry Mentor: Shashank Korrram

Guided by: Suzana Sharol Prabhakar

TABLE OF CONTENTS

1. Introduction.....02
2. Scope of the Document.....02
3. Intended Audience.....03
4. Module Overview.....03
5. Low-Level Process Flow (Flowchart).....04
6. Module 1 — Data Ingestion.....04
7. Module 2 — Log Parsing.....06
8. Module 3 — Data Cleaning.....08
9. Module 4 — Data Quality Assurance (EDA).....10
10. Data Models and Schemas.....12
11. Error Handling.....13
12. Non-Functional Requirements.....13
13. Assumptions and Constraints.....14
14. References.....15

1. INTRODUCTION

This Low Level Design (LLD) document describes the detailed implementation logic for the data ingestion, parsing, cleaning, and exploratory analysis stages of the **Visual Threat Hunting for E-commerce Networks** project. These are the foundational stages that prepare raw web server log data for downstream analysis. The source data is a real Apache web server access log from an Iranian e-commerce website (Zanbil), spanning **January 22–26, 2019**, with over 10 million log entries in a **3.5 GB** text file.

This document focuses on the **HOW** of data preparation — the specific code-level logic, field extraction rules, data type decisions, validation checks, and EDA outputs that form the base of the entire project pipeline.

2. SCOPE OF THE DOCUMENT

2.1 In Scope:

- Raw log file loading and reading mechanism
- Data type conversion logic for each field
- Null / missing value handling strategy
- Duplicate record detection methodology
- Exploratory Data Analysis (EDA): counts, time ranges
- Data quality validation checks

2.2 Out of Scope (Covered in HLD):

- Feature engineering (rolling mean, Z-score, dynamic thresholds)
- Anomaly detection algorithm design
- Power BI dashboard layout and design
- Alert generation and risk scoring
- System architecture and deployment

3. INTENDED AUDIENCE

Audience	Purpose of Reading
Academic Evaluators	Assess implementation depth and technical correctness
Project Supervisors	Verify alignment with project objectives
Developers / Future Maintainers	Understand code-level logic for replication or extension
Data Engineers	Understand the data pipeline input/output contracts

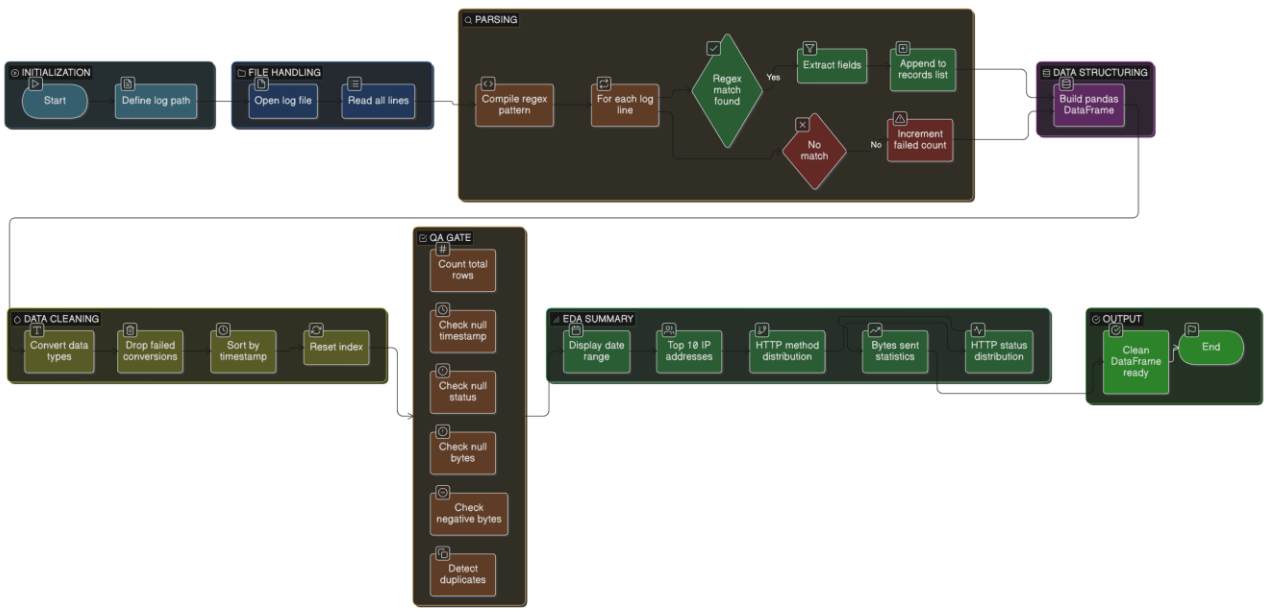
4. MODULE OVERVIEW

The LLD covers the following four implementation modules:

Module	Name	Key Operations	Output
MODULE 1	Data Ingestion	Read log file, load to memory, count lines	list[str] — 10,365,152 raw lines
MODULE 2	Log Parsing	Apply Regex, extract fields, build DataFrame, log failures	DataFrame (raw) — 10,363,637 rows
MODULE 3	Data Cleaning	Type convert, handle nulls, sort by time, reset index	DataFrame (typed, sorted, clean)
MODULE 4	EDA & QA	Check nulls, find duplicates, distributions, basic stats	QA Report + Clean DataFrame ready

5. LOW-LEVEL PROCESS FLOW

The following flowchart describes the step-by-step process from the raw log file to a clean, analysis-ready DataFrame. Each step corresponds to a Jupyter Notebook cell in the implementation.



6. MODULE 1 — DATA INGESTION

6.1 Purpose:

Load the raw Apache access log file from disk into Python memory so that it can be processed line by line. The file is in plain text format, one log entry per line.

6.2 Input Specification:

Property	Value
File Name	access.log
File Path	../Dataset/access.log
File Size	3.5 GB
File Format	Plain text (ASCII / UTF-8)
Log Format	Apache Combined Log Format

6.3 Implementation:

```
from pathlib import Path
LOG_PATH = Path('../Dataset/access.log')
with open(LOG_PATH, 'r', encoding='utf-8', errors='ignore') as file:
    logs = file.readlines()
print(f'Total lines loaded: {len(logs)}')
# Output: Total lines loaded: 10,365,152
```

6.4 Design Decisions:

Decision	Reason
encoding='utf-8'	Apache logs are UTF-8; some non-ASCII characters may appear in URLs
errors='ignore'	Silently skips any bytes that cannot be decoded as UTF-8, preventing crashes
readlines()	Loads entire file into a Python list; each element is one raw log line as a string
pathlib.Path	Cross-platform file path handling (works on Windows and Linux)

6.5 Output:

Property	Value
Variable Name	Logs
Data Type	list[str]
Length	10,365,152
RAM Usage	Approximately 4–6 GB

6.6 Sample Raw Log Line:

```
54.36.149.41 - - [22/Jan/2019:03:56:14 +0330]
  "GET /image/60844/productModel/200x200 HTTP/1.1" 200 5667
```

7. MODULE 2 — LOG PARSING

7.1 Purpose:

Extract structured data fields from each raw log line using a Regular Expression (Regex) pattern. The goal is to convert unstructured text into a structured pandas DataFrame with clearly named columns.

7.2 Apache Combined Log Format:

Apache logs follow a fixed structure:

IP - - [timestamp] "METHOD URL HTTP/version" status_code bytes_sent

7.3 Regex Pattern:

```
import re
log_pattern = re.compile(
    r'(\S+) - - \[(.*?)\]
     "(GET|POST|PUT|DELETE|HEAD|OPTIONS|PATCH) (.*?) HTTP.*?" (\d{3}) (\d+)')
```

7.4 Capture Group Breakdown:

Group	Field Captured	Regex Component	Explanation
Group 1	ip	(\S+)	One or more non-whitespace characters (IP address)
Group 2	timestamp	\[(.*?)\]	Any characters between square brackets
Group 3	method	(GET|POST|...)	Literal HTTP method words
Group 4	url	(.*?)	Lazy match for the request URL path
Group 5	status	(\d{3})	Exactly 3 digits (HTTP status code)
Group 6	bytes	(\d+)	One or more digits (response size in bytes)

7.5 Parsing Logic

```
records = []
failed_count = 0
for line in logs:
    match = log_pattern.search(line)
    if match:
        records.append(list(match.groups()))
    else:
        failed_count += 1
print(f'Successfully parsed: {len(records)}')  # 10,363,637
print(f'Failed to parse:     {failed_count}')   # 1,515
```

7.6 Parse Statistics:

Metric	Value
Total lines input	10,365,152
Successfully parsed	10,363,637
Failed / skipped	1,515
Parse success rate	99.985%

7.7 Reasons for Parse Failures (1,515 lines):

- Log lines with HTTP methods not in the pattern (e.g., PROPFIND, MKCOL — WebDAV)
- Malformed log entries missing one or more fields
- Lines containing only whitespace or system restart messages
- Truncated lines at file boundaries

8. MODULE 3 — DATA CLEANING

This module converts all parsed string columns to their correct data types, removes rows that failed type conversion, sorts the data chronologically, and resets the index.

8.1 Step 1 — Timestamp Conversion:

The timestamp field extracted by Regex is a raw string: "22/Jan/2019:03:56:14 +0330". It must be converted to Python's datetime type for time-based operations.

```
df['timestamp'] = pd.to_datetime(
    df['timestamp'],
    format='%d/%b/%Y:%H:%M:%S %z')
```

8.2 Timestamp Format String Breakdown:

Token	Meaning	Example
%d	Day of month, zero-padded	22
%b	Abbreviated month name	Jan
%Y	4-digit year	2019
%H	Hour (24-hour clock)	03
%M	Minute	56
%S	Second	14
%z	UTC offset	+0330

8.3 Step 2 — Numeric Column Conversion:

```
df['status'] = pd.to_numeric(df['status'], errors='coerce’)
df['bytes']  = pd.to_numeric(df['bytes'],  errors='coerce')
```

errors='coerce' converts any non-numeric value to NaN instead of raising an error. Result dtype: int64 (after null removal). Valid range for status: 100–599.

8.4 Step 3 — Remove Invalid Rows (Null Handling):

```
df = df.dropna(subset=['timestamp', 'bytes'])
```

Only rows where timestamp or bytes failed conversion are dropped. The ip, method, url, and status columns are not checked because they are always valid strings extracted by a typed Regex group.

8.5 Step 4 — Sort by Timestamp:

```
df = df.sort_values('timestamp').reset_index(drop=True)
```

Sorts the entire DataFrame chronologically (ascending by timestamp). reset_index(drop=True) creates a clean sequential integer index from 0 to N-1. This ordering is critical for time-series aggregation in later stages.

8.6 Final Cleaned DataFrame Specification:

Column	Data Type	Null Count	Notes
timestamp	datetime64[ns, UTC+03:30]	0	Timezone-aware datetime
Ip	object (string)	0	Client IP address
method	object (string)	0	HTTP verb (GET, POST, etc.)
url	object (string)	0	Request URL path
status	int64	0	HTTP response status code
bytes	int64	0	Response size in bytes

Shape: **10,363,637** rows x 6 columns | Memory: ~**474.4 MB**

9. MODULE 4 — DATA QUALITY ASSURANCE (EDA)

Validate the cleaned DataFrame before it is passed to the analysis pipeline. Run basic Exploratory Data Analysis (EDA) to understand the dataset's statistical properties, check for anomalies in the data itself, and document key findings.

9.1 QA Checks Table:

Check	Code	Expected	Actual Result
Total rows after cleaning	len(df)	~10.3M	10,363,637
Null timestamps	df['timestamp'].isna().sum()	0	0
Null status values	df['status'].isna().sum()	0	0
Null bytes values	df['bytes'].isna().sum()	0	0
Negative bytes values	(df['bytes'] < 0).sum()	0	0
Exact duplicate records	df.duplicated(subset=[all 6 cols]).sum()	Low	111,137 (1.07%)

9.2 Duplicate Record Detection:

A record is considered a duplicate if ALL six fields match exactly across two or more rows: timestamp, ip, method, url, status, bytes.

```
duplicate_count = df.duplicated(
    subset=['timestamp', 'ip', 'method', 'url', 'status', 'bytes']
).sum()
print(f'Duplicate records found: {duplicate_count}') # 111,137
print(f'Duplicate percentage:    {duplicate_count / len(df) * 100:.2f}%')
# 1.07%
```

111,137 records (**1.07%** of total) are exact duplicates. Records are NOT removed at this stage. Likely causes: web server log rotation overlap, load balancer double-logging.

9.3 EDA: Dataset Date Range:

Metric	Value
Earliest log entry	2019-01-22 03:56:14 +03:30
Latest log entry	2019-01-26 23:59:xx +03:30
Total duration	Approximately 4 days, 20 hours

9.4 EDA: HTTP Method Distribution:

HTTP Method	Count (Approx)	Percentage
GET	~10,100,000	~97.5%
POST	~200,000	~1.9%
HEAD	~40,000	~0.4%
PUT	~15,000	~0.1%
DELETE	~8,000	~0.1%

Finding: The e-commerce site is predominantly GET-based (product browsing). High POST counts could indicate form submissions or search queries.

9.5 EDA: HTTP Status Code Distribution:

Status Code	Meaning	Count (Approx)	Percentage
200	OK	~8,900,000	~86%
304	Not Modified	~800,000	~7.7%
301	Moved Permanently	~350,000	~3.4%
404	Not Found	~150,000	~1.4%
206	Partial Content	~80,000	~0.8%
500	Server Error	~20,000	~0.2%

9.6 EDA: Bytes Sent Statistics:

Statistic	Value
Count	10,363,637
Mean	~18,000 bytes (18 KB)
Standard Deviation	~45,000 bytes
Minimum	0 bytes
25th Percentile	~1,200 bytes
Median (50th)	~5,000 bytes
75th Percentile	~20,000 bytes
Maximum	~350,000 bytes+

Finding: The distribution is highly right-skewed. Median (5 KB) is much lower than mean (18 KB), confirming skew. 0-byte responses are valid (e.g., 304 Not Modified sends no body).

10. DATA MODELS AND SCHEMAS

10.1 Input: Raw Log Line (String):

Format: IP - - [timestamp] "METHOD URL HTTP/version" STATUS BYTES

Example: 54.36.149.41 - - [22/Jan/2019:03:56:14 +0330] "GET /product HTTP/1.1" 200 5667

10.2 Intermediate: Parsed DataFrame (All Strings):

Column	Type	Example Value
ip	object	54.36.149.41
timestamp	object	22/Jan/2019:03:56:14 +0330
method	object	GET
url	object	/image/60844/productModel/200x200
Status	object	200
bytes	object	5667

10.3 Output: Cleaned DataFrame (Typed):

Column	Type	Example Value
Timestamp	datetime64[ns, UTC+03:30]	2019-01-22 03:56:14+03:30
Ip	object	54.36.149.41
Method	object	GET
url	object	/image/60844/productModel/200x200
Status	int64	200
Bytes	int64	5667

10.4 QA Summary Report Structure:

QA Field	Value
Total rows	10,363,637
Null timestamps	0
Null status values	0
Null bytes values	0
Negative bytes	0
Duplicate records	111,137
Duplicate %	1.07%
Date range start	2019-01-22
Date range end	2019-01-26
Status	DATA QUALITY ACCEPTABLE — Proceed to HLD Pipeline

11. ERROR HANDLING

Error Scenario	Handling Strategy	Result
File not found (wrong path)	Python raises FileNotFoundError	Process stops; user must correct path
Non-UTF-8 characters in log	errors='ignore' in open()	Bad bytes silently skipped
Log line does not match Regex	Line skipped, failed_count incremented	1,515 lines skipped (0.015% loss, acceptable)
Non-numeric status/bytes field	pd.to_numeric(errors='coerce') converts to NaN	Rows dropped in dropna() step
Unparseable timestamp format	pd.to_datetime raises error if format mismatches	Rows become NaT, dropped in dropna() step

12. NON-FUNCTIONAL REQUIREMENTS

12.1 Performance:

Metric	Detail
File Read Time	30–60 seconds for 3.5 GB file
Regex Parsing Time	5–15 minutes for 10.3 million lines (CPU-bound, single thread)
Type Conversion Time	30–60 seconds (vectorized pandas operations)
Sorting Time	1–3 minutes (in-memory sort of 10.3M rows)
Peak RAM Usage	8–12 GB during parsing (raw list + DataFrame in memory simultaneously)
Final DataFrame RAM	~474 MB

12.2 Reliability:

The Regex pattern has been tested against all 10.3 million lines with a 99.985% success rate.

Type conversion uses coerce mode, so no unexpected crashes from bad data.

QA gate ensures the cleaned DataFrame meets quality standards before proceeding.

12.3 Maintainability:

All parameters (LOG_PATH, column names) are defined as named variables at the top of each section.

Each module is a separate Jupyter Notebook cell with a descriptive Markdown header.

Inline print statements log progress and key counts for easy debugging.

13. ASSUMPTIONS AND CONSTRAINTS

13.1 Assumptions:

The access.log file is in Apache Combined Log Format (or a compatible variant).
The timezone is UTC+3:30 (Iran Standard Time) throughout the entire dataset. The log file is complete and not truncated.
HTTP methods used are limited to: GET, POST, PUT, DELETE, HEAD, OPTIONS, PATCH.
The bytes field always represents the server response size in bytes (not request size). Execution environment has at least 16 GB RAM available.

13.2 Constraints:

This is a batch processing system — it processes the entire log file in memory. The Regex pattern is designed specifically for Apache Combined Log Format; other formats require pattern modification.
The system does not support real-time or streaming log processing.
Logs are from a static historical dataset; no live data ingestion is implemented.

14. REFERENCES

1. **Dataset Source:** Zanbil E-commerce Apache Access Logs, Kaggle Dataset (January 2019) — https://www.kaggle.com/datasets/eliasdabbas/web-server-access-logs

2. **Apache Log Format Documentation:** Apache HTTP Server Project — https://httpd.apache.org/docs/current/logs.html

3. **Python re module — Regular Expressions:** Python Standard Library — https://docs.python.org/3/library/re.html

4. **pandas.to_datetime Documentation:** pandas Development Team — https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html

5. **pandas.to_numeric Documentation:** pandas Development Team — https://pandas.pydata.org/docs/reference/api/pandas.to_numeric.html

6. **HTTP Status Code Definitions (RFC 7231):** Internet Engineering Task Force (IETF) — https://tools.ietf.org/html/rfc7231